

Le Perceptron

Le Premier Neurone Artificiel

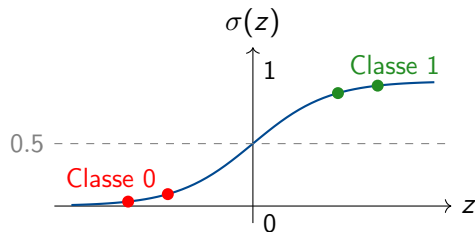
Plan du Cours

- 1 Contexte
- 2 Le Modèle Mathématique
- 3 Interprétation Géométrique
- 4 L'Algorithme d'Apprentissage
- 5 Limitations : Le Problème XOR
- 6 Extension Multi-Classes
- 7 Et Après ?

Rappel : La Régression Logistique

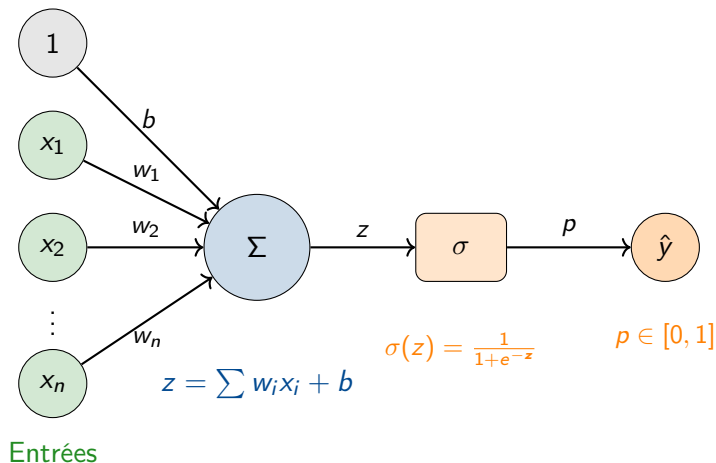
La Régression Logistique :

- Combinaison linéaire : $z = w^T x + b$
- Fonction **sigmoïde** : $\sigma(z) = \frac{1}{1+e^{-z}}$
- Produit une **probabilité** $\in [0, 1]$
- Seuil à 0.5 pour décider la classe



Un modèle élégant avec une **mesure de confiance**.

Rappel : Architecture du Neurone Logistique



La sortie est une **probabilité** : on peut dire "je suis sûr à 87% que c'est la classe 1".

Un modèle plus ancien mais fondamental :

Le Perceptron

L'ancêtre direct des réseaux de neurones modernes

De la Régression Logistique au Perceptron

Le **Perceptron** est similaire, mais avec une approche plus brutale :

Points Communs

- Combinaison linéaire : $z = w^T x + b$
- Classifieur binaire
- Frontière de décision linéaire

Différences

- **Pas de probabilité** : sortie directe 0 ou 1
- Fonction d'activation en **escalier** (step function)
- Mise à jour des poids **uniquement sur les erreurs**

Qu'est-ce que le Perceptron ?

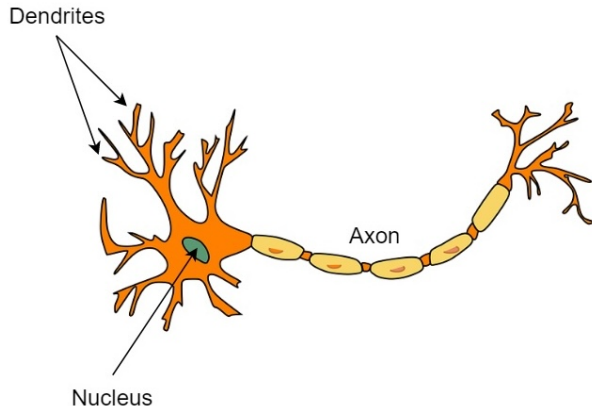
Définition

Le Perceptron est l'un des types de neurones artificiels les plus simples et les plus anciens, développé par **Frank Rosenblatt en 1957**.

Caractéristiques :

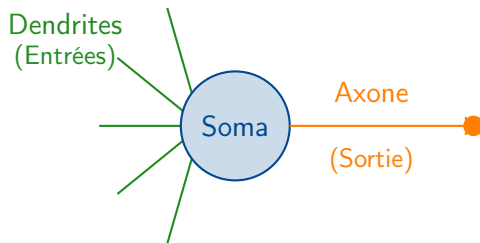
- Algorithme de **classification binaire**
- Examine des données d'entrée et décide entre deux classes
- Exemples : "Oui/Non", "Spam/Pas Spam", "Chat/Chien"

Le Perceptron est le "Hello World" des réseaux de neurones.



Analogie

- 1 Reçoit des signaux électriques (**Entrées**) via les dendrites
- 2 Traite la force de ces signaux (**Poids**)
- 3 Si le signal total dépasse un seuil, le neurone "s'active" (**Activation**)



Analogie

- 1 Reçoit des signaux électriques (**Entrées**) via les dendrites
- 2 Traite la force de ces signaux (**Poids**)
- 3 Si le signal total dépasse un seuil, le neurone "s'active" (**Activation**)

Le Modèle Mathématique

Les Composants du Perceptron

❶ Entrées x : Caractéristiques des données

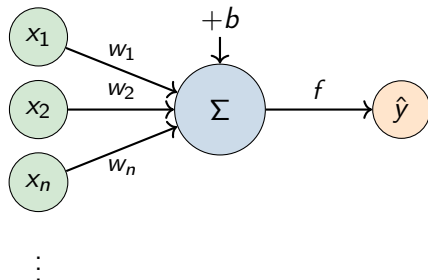
- Vecteur $x = (x_1, x_2, \dots, x_n)$

❷ Poids w : Importance de chaque entrée

- Vecteur $w = (w_1, w_2, \dots, w_n)$

❸ Biais b : Valeur de décalage

- Similaire à l'ordonnée à l'origine



Calcul de la Somme Pondérée z

$$z = \sum_{i=1}^n w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$

Interprétation :

- Chaque entrée x_i est multipliée par son poids w_i
- Plus le poids est grand (en valeur absolue), plus l'entrée a d'importance
- Le biais b permet de décaler le seuil d'activation

Exemple

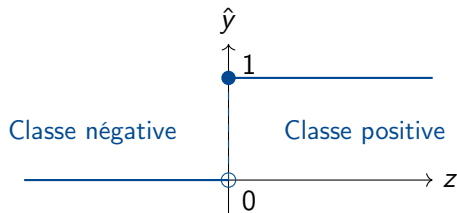
Pour $\mathbf{x} = (2, 3)$, $\mathbf{w} = (0.5, -1)$, $b = 1$:

$$z = 0.5 \times 2 + (-1) \times 3 + 1 = 1 - 3 + 1 = -1$$

La Fonction d'Activation : Step Function

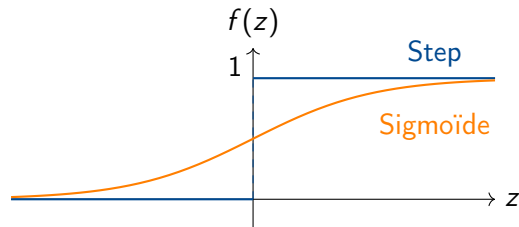
Fonction de Heaviside (Fonction en Escalier)

$$\hat{y} = f(z) = \begin{cases} 1 & \text{si } z \geq 0 \\ 0 & \text{si } z < 0 \end{cases}$$



Pas de nuance : la décision est **binaire et immédiate**.

Comparaison : Step vs Sigmoid



Step Function

- Transition brutale
- Non différentiable en 0
- Sortie : 0 ou 1

Sigmoide

- Transition douce
- Différentiable partout
- Sortie : probabilité $[0, 1]$

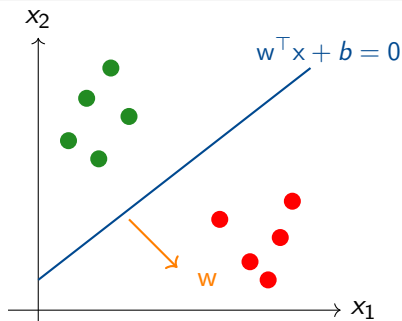
Interprétation Géométrique

Le Perceptron comme Classifieur Linéaire

Frontière de Décision

L'équation $w^T x + b = 0$ définit une **frontière linéaire** :

- En **2D** : une droite
- En **3D** : un plan
- En **dimension n** : un hyperplan



Le Rôle du Vecteur de Poids

Le vecteur w

- w est **perpendiculaire** à la frontière de décision
- Il pointe vers la région des exemples **positifs**

Le biais b

- Contrôle la **distance** de la frontière à l'origine
- $b > 0$: la frontière s'éloigne de l'origine
- $b < 0$: la frontière se rapproche de l'origine

Le travail du Perceptron : Faire pivoter et déplacer cette ligne jusqu'à séparer parfaitement les deux classes.

L'Algorithme d'Apprentissage

Comment le Perceptron Apprend

Principe Fondamental

Le Perceptron apprend **en faisant des erreurs** et en se corrigeant.

Les Étapes de l'Algorithme

- ➊ **Initialiser** : Commencer avec w et b aléatoires (ou nuls)
- ➋ **Prédire** : Pour un point $x^{(i)}$, calculer $\hat{y}^{(i)} = f(w^\top x^{(i)} + b)$
- ➌ **Comparer** : Vérifier si $\hat{y}^{(i)} = y^{(i)}$ (étiquette réelle)
- ➍ **Mettre à jour** : Si erreur, ajuster w et b
- ➎ **Répéter** : Parcourir tous les exemples jusqu'à convergence

La Règle de Mise à Jour

Formule de Mise à Jour

Si la prédiction est **incorrecte** :

$$w \leftarrow w + \eta \cdot (y^{(i)} - \hat{y}^{(i)}) \cdot x^{(i)}$$

$$b \leftarrow b + \eta \cdot (y^{(i)} - \hat{y}^{(i)})$$

où η est le **taux d'apprentissage** (ex : 0.01, 0.1).

Intuition

- Si $\hat{y} = 0$ mais $y = 1$: erreur = +1 $\Rightarrow$ On **ajoute** x aux poids
- Si $\hat{y} = 1$ mais $y = 0$: erreur = -1 $\Rightarrow$ On **soustrait** x des poids

Parallèle avec la Descente de Gradient

On connaît déjà la **descente de gradient** :

$$w \leftarrow w - \eta \cdot \nabla_w J(w)$$

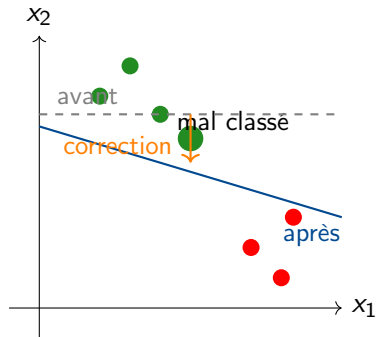
Rappel : η = taux d'apprentissage, $\nabla_w J$ = gradient de la fonction de coût J

	Descente de Gradient	Perceptron
Mise à jour	Sur tous les exemples	Uniquement sur erreurs
Différentiable ?	Oui (requis)	Non (step function)

Le Lien

La règle du Perceptron peut être vue comme une **SGD** sur une fonction de perte particulière. Elle a été inventée avant (1957) !

Visualisation de l'Apprentissage



À chaque erreur, la frontière pivote pour mieux classer le point mal classé.

Exemple : Entraînement Pas à Pas

Données : $x^{(1)} = (1, 2), y^{(1)} = 1$ $x^{(2)} = (2, 1), y^{(2)} = 0$ $\eta = 1$

Initialisation : $w = (w_1, w_2) = (0, 0), b = 0$

Étape 1 — Point $x^{(1)} = (x_1^{(1)}, x_2^{(1)}) = (1, 2), y^{(1)} = 1$

- $z = w_1 x_1^{(1)} + w_2 x_2^{(1)} + b = 0 \cdot 1 + 0 \cdot 2 + 0 = 0 \Rightarrow \hat{y} = 1$
 - $\hat{y} = y^{(1)}$? **✓ Oui !** $\Rightarrow$ Pas de mise à jour
 - $w = (0, 0), b = 0$
-

Étape 2 — Point $x^{(2)} = (x_1^{(2)}, x_2^{(2)}) = (2, 1), y^{(2)} = 0$

- $z = w_1 x_1^{(2)} + w_2 x_2^{(2)} + b = 0 \cdot 2 + 0 \cdot 1 + 0 = 0 \Rightarrow \hat{y} = 1$
- $\hat{y} = y^{(2)}$? **✗ Non !** $\Rightarrow$ Mise à jour !
- $w \leftarrow (0, 0) + 1 \cdot (0 - 1) \cdot (2, 1) = (-2, -1)$
- $b \leftarrow 0 + 1 \cdot (0 - 1) = -1$

Exemple : Entraînement Pas à Pas (suite)

État actuel : $w = (w_1, w_2) = (-2, -1)$, $b = -1$

Étape 3 — Point $x^{(1)} = (x_1^{(1)}, x_2^{(1)}) = (1, 2)$, $y^{(1)} = 1$

- $z = w_1 x_1^{(1)} + w_2 x_2^{(1)} + b = (-2) \cdot 1 + (-1) \cdot 2 + (-1) = -5 \Rightarrow \hat{y} = 0$
 - $\hat{y} = y^{(1)}$? **Non !** $\Rightarrow$ Mise à jour !
 - $w \leftarrow (-2, -1) + 1 \cdot (1 - 0) \cdot (1, 2) = (-1, 1)$
 - $b \leftarrow -1 + 1 \cdot (1 - 0) = 0$
-

Étape 4 — Point $x^{(2)} = (x_1^{(2)}, x_2^{(2)}) = (2, 1)$, $y^{(2)} = 0$

- $z = w_1 x_1^{(2)} + w_2 x_2^{(2)} + b = (-1) \cdot 2 + 1 \cdot 1 + 0 = -1 \Rightarrow \hat{y} = 0$
- $\hat{y} = y^{(2)}$? **Oui !** $\Rightarrow$ Pas de mise à jour

Continuer jusqu'à ce que tous les points soient correctement classés (ou un maximum d'itérations).

Algorithme du Perceptron

Initialiser $w = 0$, $b = 0$

Pour chaque époque :

 Pour chaque exemple $(x^{(i)}, y^{(i)})$:

$$z = w^\top x^{(i)} + b$$

$$\hat{y} = 1 \text{ si } z \geq 0 \text{ sinon } 0$$

 Si $\hat{y} \neq y^{(i)}$:

$$w \leftarrow w + \eta(y^{(i)} - \hat{y})x^{(i)}$$

$$b \leftarrow b + \eta(y^{(i)} - \hat{y})$$

Jusqu'à convergence (0 erreur) ou max itérations

Théorème de Convergence du Perceptron

Garantie Théorique

Théorème : Si les données sont **linéairement séparables**, l'algorithme du Perceptron converge en un nombre fini d'étapes.

Implications :

- Après suffisamment d'itérations, le Perceptron trouve une frontière qui sépare parfaitement les classes

Attention

Si les données **ne sont pas** linéairement séparables, l'algorithme **ne converge pas**.

Les Limitations

Le Problème du XOR

Le Défi

Un Perceptron peut-il apprendre les **portes logiques** de base ?

Contexte :

- x_1 et x_2 sont des **entrées binaires** : $x_1, x_2 \in \{0, 1\}$
- La sortie est aussi binaire : $y \in \{0, 1\}$
- Il n'y a que **4 combinaisons possibles** : $(0, 0), (0, 1), (1, 0), (1, 1)$

Question

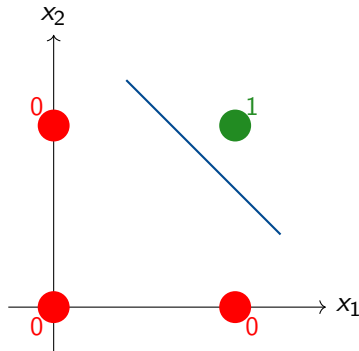
Pour chaque porte logique, peut-on tracer **une droite** qui sépare les sorties 0 des sorties 1 ?

Porte ET (AND)

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

Règle : Sortie = 1 seulement si **les deux** entrées sont 1.

✓ **Linéairement séparable** : une droite peut isoler le seul point vert.

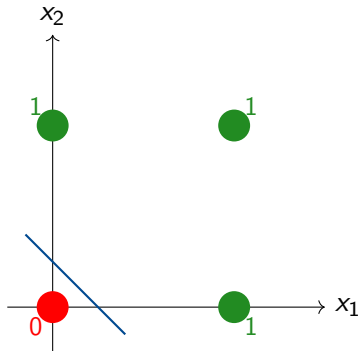


Porte OU (OR)

x_1	x_2	OR
0	0	0
0	1	1
1	0	1
1	1	1

Règle : Sortie = 1 si **au moins une** entrée est 1.

✓ **Linéairement séparable** : une droite peut isoler le seul point rouge.

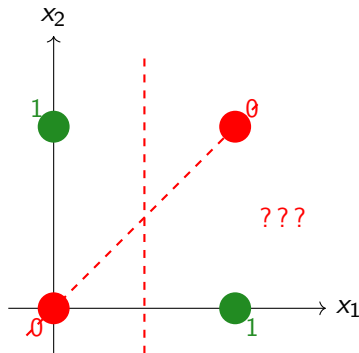


Porte XOR (OU Exclusif)

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Règle : Sortie = 1 si **exactement une** entrée est 1.

× **NON linéairement séparable** : impossible de tracer une seule droite !



La Critique de Minsky & Papert (1969)

Dans leur livre "*Perceptrons*", ils démontrent mathématiquement que le Perceptron **ne peut pas** résoudre le problème XOR (et d'autres problèmes non-linéaires).

Conséquences :

- Réduction drastique des financements pour la recherche en IA
- Période connue comme le premier "**Hiver de l'IA**" (années 1970)
- Renaissance dans les années 1980 avec les **réseaux multicouches**

La Leçon

Un seul neurone = une seule frontière linéaire = **capacité limitée**

Résumé du Perceptron Simple

Caractéristique	Description
Type	Apprentissage Supervisé, Classifieur Binaire
Mathématiques	$z = w^T x + b$
Activation	Fonction de Heaviside (Marche)
Apprentissage	Mise à jour uniquement en cas d'erreur
Limite Clé	Échoue si données non linéairement séparables

Le Perceptron est Binaire... Et si on a K classes ?

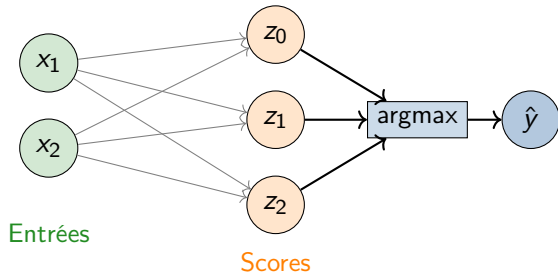
Le Perceptron classique distingue **2 classes** : $\hat{y} \in \{0, 1\}$

Comment classer parmi K classes ? (ex : chiffres 0-9, $K = 10$)

Solution : Perceptron Multi-Sorties

Utiliser K neurones de sortie, un par classe. Chaque neurone calcule un **score** pour sa classe.

Prédiction : $\hat{y} = \arg \max_k z_k$ (la classe avec le score le plus élevé)



Formule matricielle :

$$z = Wx + b$$

où :

- W : matrice $K \times n$
- x : vecteur $n \times 1$
- b : vecteur $K \times 1$
- z : vecteur $K \times 1$

$$\hat{y} = \arg \max_k z_k$$

Calcul Matriciel : Formule Générale

Pour $K = 3$ classes et $n = 2$ entrées :

$$z = Wx + b$$

$$\begin{pmatrix} z_0 \\ z_1 \\ z_2 \end{pmatrix} = \begin{pmatrix} w_{0,1} & w_{0,2} \\ w_{1,1} & w_{1,2} \\ w_{2,1} & w_{2,2} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} b_0 \\ b_1 \\ b_2 \end{pmatrix}$$

• Classe 0 • Classe 1 • Classe 2

Développement :

$$z_0 = w_{0,1}x_1 + w_{0,2}x_2 + b_0$$

$$z_1 = w_{1,1}x_1 + w_{1,2}x_2 + b_1$$

$$z_2 = w_{2,1}x_1 + w_{2,2}x_2 + b_2$$

Chaque **ligne** = une classe

Exemple Numérique : 3 Classes, 2 Entrées

Données : $x = \begin{pmatrix} 2 \\ 1 \end{pmatrix}$

Poids :

$$W = \begin{pmatrix} 1 & -1 \\ 0.5 & 2 \\ -1 & 0.5 \end{pmatrix} \quad b = \begin{pmatrix} 0 \\ 0.5 \\ 1 \end{pmatrix}$$

Calcul :

$$z = \begin{pmatrix} 1 & -1 \\ 0.5 & 2 \\ -1 & 0.5 \end{pmatrix} \begin{pmatrix} 2 \\ 1 \end{pmatrix} + \begin{pmatrix} 0 \\ 0.5 \\ 1 \end{pmatrix}$$

Résultat :

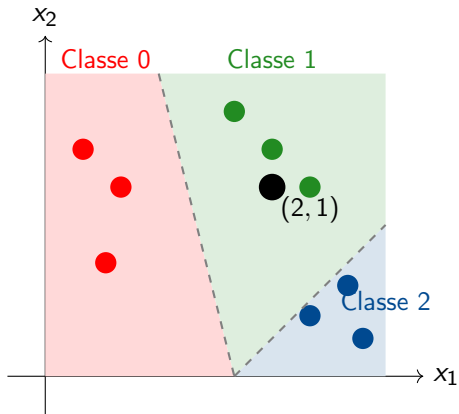
$$z = \begin{pmatrix} 2 - 1 \\ 1 + 2 \\ -2 + 0.5 \end{pmatrix} + \begin{pmatrix} 0 \\ 0.5 \\ 1 \end{pmatrix} = \begin{pmatrix} 1 \\ 3.5 \\ -0.5 \end{pmatrix}$$

Scores :

- Classe 0 : $z_0 = 1$
- Classe 1 : $z_1 = 3.5 \leftarrow \max$
- Classe 2 : $z_2 = -0.5$

$$\hat{y} = \arg \max(z) = 1$$

Visualisation : Frontières Multi-Classes



Chaque classe a sa région. Les frontières sont des **droites** (linéaires).

Algorithme

Pour un exemple (x, y) où y est la vraie classe :

- ① Calculer $\hat{y} = \arg \max_k z_k$
- ② Si $\hat{y} \neq y$ (erreur) :
 - **Renforcer** la bonne classe : $w_y \leftarrow w_y + \eta x$
 - **Pénaliser** la mauvaise prédiction : $w_{\hat{y}} \leftarrow w_{\hat{y}} - \eta x$

Intuition

On "pousse" les poids pour que le score de la vraie classe devienne plus grand que celui de la classe prédite.

*Le Perceptron (même multi-classes) reste **linéaire**.*

Comment surmonter cette limitation ?



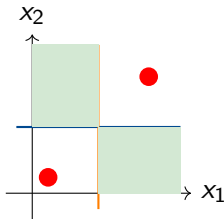
Le Perceptron Multicouche (MLP)

La Solution : Combiner les Neurones

L'idée

Si un neurone trace une ligne, que se passe-t-il si nous **combinons plusieurs neurones** ?

- Un neurone $\rightarrow$ une ligne
- Deux neurones $\rightarrow$ deux lignes
- Combiner leurs sorties $\rightarrow$ formes complexes !



Deux lignes peuvent isoler les points positifs du XOR !